

Backprop versus the memory wall

Compute-to-commute heuristics and the batch-size window, on an A100 and on Dally’s 2D grid

11 September 2026

Companion files: `backprop_memory_wall.py` regenerates every table and the figure from the parameters listed in the appendix; `fig_backprop_memory_wall.png` is the three-panel figure. All energy numbers are stated in picojoules and in Dally’s unit, the int8 add (8 fJ at 1 fJ/bit), so they can be re-derived under a different wire-energy convention by changing one constant.

0. Summary

The unit. Your unit is exactly Dally’s 2023 unit: an add costs about 1 fJ/bit and on-chip wire about 100 fJ/bit·mm, so *one add is worth 10 μ m of movement* and one bit across a 16 mm die is worth 1,600 adds. In that unit, on an A100: a byte from HBM costs $\sim 5,000$ adds at the memory interface and $\sim 7,500$ all-in (add the on-die round trip); a byte from the 40 MB L2 $\sim 1,500$; a byte from shared memory ~ 120 ; and a BF16 FLOP costs ~ 12 adds in the datapath but ~ 110 adds all-in, because the other ~ 100 are operand delivery and orchestration. Arithmetic is the cheapest thing a training step does.

Eleven heuristics (all derived below; numbers are A100-80GB unless stated):

1. **Reuse rule.** A byte fetched across boundary i must be reused at least β_i times, where β_t = peak/bandwidth (time) and β_E = $e_{\text{byte}}/e_{\text{FLOP}}$ (energy). At HBM: β_t = 153 FLOP/B, β_E = 67 (A100); 295/91 (H100). β_t roughly doubles per GPU generation while on-chip capacities do not, so tiles *and* batches must keep growing.
2. **Training-step intensity.** At the HBM boundary, $I(T) \approx \frac{72Td}{12b_wd + aT}$: it is $\approx T/2$ while weights are being streamed and saturates at $\approx 72d/a \approx 0.36d$ (about 10^3 FLOP/B at $d = 4096$ once L2 re-blocking is included).
3. **Batch floor.** $T_{\min} = \frac{12b_wd\beta_t}{72d - a\beta_t} \rightarrow 2\beta_t \approx 300$ tokens per weight visit on A100, ≈ 600 on H100, diverging at $d_{\min} = a\beta_t/72 \approx 450$ (A100) / 900 (H100). Below $d_{\min}$ no batch size helps; only fusion does.
4. **Rank rule.** A micro-batch of T tokens carries at most rank- T information per weight matrix. Moving W or ∇W (dk numbers) instead of the factors $X, \nabla Y$ ($T(d+k)$ numbers) is redundant while $T < dk/(d+k) \approx 0.8d$. The same “ $T \approx d$ ” threshold falls out from the information side as from the bandwidth side.
5. **Element-wise time tax** $\approx a\beta_t/(72d) \approx 2.8\beta_t/d$ of GEMM time: 10% at $d = 4096$, 55% at $d = 768$ on A100, 107% at $d = 768$ on H100. The corresponding *energy* tax is only $a e_B/(72 d e_F) \approx 4.5\%$ at $d = 4096$. Backprop’s activation round trip is cheap in joules and expensive in seconds.
6. **Store versus recompute.** Round-tripping a token-layer’s saved activations through HBM costs $68 d e_B$; recomputing the forward costs $24 d^2 e_F$. Storing wins for $d > 2.8 e_B/e_F \approx$

190. Gradient checkpointing therefore buys *capacity* at +33% arithmetic energy; it lowers no traffic.
7. **Per-step fixed traffic.** Adam in mixed precision moves ~ 30 B/param/step: parity with compute at $T_{\text{step}} \approx 5\beta_t \approx 800$ tokens per device, <10% overhead at $\sim 10^4$. Data-parallel ring all-reduce moves ~ 4 B/param over the fabric: $T_{\text{dev}} \geq \beta_{\text{net}}/1.5 \approx 700$ (NVLink) or $\approx 8,000$ (IB HDR) per step. ZeRO-3/FSDP moves ~ 6 B/param *per micro-batch*: $T_{\text{dev}} \geq \beta_{\text{net}}$ per micro-batch.
 8. **Model-parallel escape.** Tensor parallelism’s fabric intensity is $4.5d/p$ FLOP/B and pipeline parallelism’s is $18dL/p$; both are batch-independent. Only data parallelism’s is $\propto T$.
 9. **The squeeze.** With McCandlish’s $S = S_{\text{min}}(1 + B_{\text{crit}}/B)$ and a fixed per-step cost worth T_{fixed} token-equivalents per device, the time-optimal global batch is $B_{\text{opt}} = \sqrt{B_{\text{crit}} N T_{\text{fixed}}}$, the geometric mean of the statistical ceiling and the hardware floor. At $N = 1024$ over IB the optimum sits *inside* the communication wall ($T_{\text{dev}} \approx 2,800 < \text{parity at } 8,300$).
 10. **On the grid.** For a weight-stationary layer on n tiles, hops per FLOP = $(d + k)\sqrt{n}/(dk)$ byte-hops, independent of batch; the backward pass is the forward pass with flow reversed (the transpose is free) and ∇W is a local outer product. What backprop costs spatially is *in-flight storage* $\propto L \times (\text{pipeline stages})$, i.e. L^2 when fully spatial, versus L for forward-only. That L^2 is the wall backprop hits on a Dally grid; reversible layers delete it for +33% FLOPs.
 11. **Single example.** Online per-token backprop of a 7B model on an A100 runs $\sim 1,000\times$ below the roofline: 25 J and 140 ms per token, 0.1% of the energy in arithmetic. Single-*sequence* backprop (2,048 tokens) is only $1.5\times$ off on one GPU (the sequence is the batch), but $5\times$ off across 64 GPUs over IB. Sixty-four sequences per GPU bring it to $1.16\times$ and ~ 50 mJ/token.

1. Compute versus commute

Definitions. For any kernel or algorithm and any boundary (register file, shared memory, L2, HBM, NVLink, InfiniBand, or a hop on a spatial grid):

- operational intensity I = FLOPs per byte crossing the boundary (Williams, Waterman & Patterson 2009);
- machine balance β_t = peak FLOP/s $\div$ bandwidth across it, and energy balance $\beta_E = e_{\text{byte}}/e_{\text{FLOP}}$;
- compute-to-commute ratio $\chi = I/\beta$; behind the wall when $\chi < 1$;
- time slowdown = $\max(1, \beta_t/I)$; commute share of energy = $\beta_E/(I + \beta_E)$.

Table 1 — the price list (A100-class, 7 nm; estimates, sources in appendix).

item	pJ	int8 adds	note
int8 add (datapath)	0.008	1	1 fJ/bit (Dally 2023)
BF16 FLOP, datapath only	0.10	12	HMMA-class MAC, 45 nm figure scaled to 7 nm
BF16 FLOP, all-in on A100 at peak	0.90	110	(400 W – 60 W static – 40 W HBM) / 312 TFLOP/s
1 byte moved 1 mm	0.8	100	100 fJ/bit-mm
1 byte across a 16 mm die	12.8	1,600	Dally’s slide
1 byte from the register file	0.3	40	
1 byte from shared memory (192 KB)	1.0	120	50 fJ/bit array + ~ 0.5 mm round trip
1 byte from L2 (40 MB)	12	1,500	array + 10-20 mm round trip

item	pJ	int8 adds	note
1 byte from HBM2e, at the interface	40	5,000	5 pJ/bit access (Dally 2023)
1 byte from HBM2e, all-in	60	7,500	+ ~25 mm on-die round trip
1 byte from HBM2e, far corner	78	9,750	+ 48 mm round trip on a 28 mm die (Dally)

Table 2 — balances at the HBM boundary.

GPU	peak TFLOP/s	HBM TB/s	β_t FLOP/B	β_E FLOP/B	$d_{\min}$	$T_{\min}(d \rightarrow \infty) = 2\beta_t$
A100-80GB	312	2.04	153	67	447	306
A100-40GB	312	1.56	201	67	595	401
H100 SXM	989	3.35	295	91	895	590
B200 (approx.)	2,250	8.0	281	136	822	562

Two things this table says that are easy to miss:

- $\beta_E < \beta_t$ on every GPU. A kernel that just reaches the roofline knee ($I = \beta_t$) still spends $\beta_E/(\beta_t + \beta_E) \approx 30\%$ of its energy on HBM bytes on A100. Time-balance and energy-balance are different targets; the energy target is easier, which is why memory-bound kernels are worse in seconds than in joules.
- The FLOP price is itself mostly commute. The all-in 0.9 pJ/FLOP is $\sim 9\times$ the datapath. It is set by tile-level reuse and by physical distance to L2, not by batch. For an $M \times N$ output tile the shared-memory intensity is $MN/(M + N)$ FLOP/B: 64 for 128×128 , 85 for 256×128 , against $\beta_{\text{SMEM}} \approx 16$ and $\beta_{\text{L2}} \approx 60$ on A100. Hopper’s L2 balance sits at or above that tile intensity, which is why it needed 4–8 $\times$ larger tiles (wgmma, TMA, clusters). This is the *interior* memory wall and no batch size touches it; only bigger tiles, fewer bits, or a spatial dataflow do.

2. The ledger of one training step

Per GPT-style layer with a $4d$ FFN: $P_\ell = 12d^2$ parameters; arithmetic $F = 72d^2$ FLOPs per token (forward $24d^2$, backward $48d^2$) plus $\approx 12sd$ for attention at sequence length s (forward $4sd$, backward $\approx 8sd$ with FlashAttention’s recompute), which is 8% of the total at $s = 2048$, $d = 4096$ and is included in the recipe.

Bytes crossing the HBM boundary, per layer:

term	bytes	per	what it is
(a) weight streaming	$b_w = 12$ per param	micro-batch	read W (bf16) in forward, again in backward for ∇X , and read-modify-write the fp32 gradient accumulator (8 B). A “lean” stack with bf16 gradients and no accumulation gets $b_w = 6$.

term	bytes	per	what it is
(b) inter-kernel activations	$a d \approx 200 d$	token	forward $\sim 52d$ (LN 4d, QKV GEMM 8d, FlashAttention 8d, out-proj + residual 6d, LN 4d, FFN-up with GeLU epilogue 10d, FFN-down + residual 12d); backward $\sim 130d$ (each op reads its saved input and ∇Y and writes ∇X , FlashAttention backward $\sim 20d$, GeLU backward $\sim 24d$). Unfused: $\sim 300d$; FP8 activations: $\sim 120d$.
(c) saved-activation capacity	$34 d$	token	Korthikanti et al. 2022, bf16, before recompute. Capacity, not traffic.
(d) L2 re-blocking	$72 d^2 / \sqrt{Z}$	token	Hong-Kung: a GEMM with MNK MACs and a cache of Z elements moves $\geq MNK / \sqrt{Z}$ elements. With $Z \approx 10^7$ (half of a 40 MB L2 in bf16), this is $93 d$ at $d = 4096$ and $280 d$ at $d = 12288$. Absent while the $T \times d$ input still fits in L2.
(e) optimizer	$b_{\text{opt}} \approx 30$ per param	step	Adam mixed precision: read fp32 master, m , v , gradient; write master, m , v , bf16 copy.
(f) fabric, data parallel	~ 4 per param	step	ring all-reduce of bf16 gradients, both directions. ZeRO-3/FSDP: ~ 6 per param per <i>micro-batch</i> (all-gather forward, all-gather backward, reduce-scatter).

Putting (a), (b), (d) together, the HBM intensity of one layer’s forward + backward as a function of tokens per weight visit T is

$$I(T) = \frac{72 d^2 T}{12 d^2 (b_w + 6T/\sqrt{Z}) + a d T} \approx \begin{cases} 6T/b_w = T/2 & T \ll d \\ \left(\frac{a}{72d} + \frac{1}{\sqrt{Z}} \right)^{-1} \approx 10^3 \text{ FLOP/B at } d = 4096 & T \gg d \end{cases}$$

The first branch is the weight-streaming regime (every byte of W is used T times, twice), the second the activation regime (traffic $\propto T$, intensity saturates). The crossover between them is at $T \approx 12 b_w d / a \approx 0.7 d$: the batch at which the bytes spent on activations equal the bytes spent on weights. That is the bandwidth-side version of the rank rule (heuristic 4).

3. Intensity versus batch: three floors and one ceiling

Floor 1 — weight reuse at HBM (per micro-batch, per weight-holding device). Solving $I(T) \geq \beta_t$:

$$T_{\min}(d) = \frac{12 b_w d \beta_t}{72 d - a_{\text{eff}} \beta_t}, \quad a_{\text{eff}} = a + \frac{72 d}{\sqrt{Z}}, \quad T_{\min} \xrightarrow{d \rightarrow \infty} \frac{b_w \beta_t}{6} = 2\beta_t.$$

Table 3 — tokens per weight visit needed to reach β_t .

d	A100-80GB	A100-40GB	H100 SXM	B200 (approx.)
512	2,520	never	never	never

d	A100-80GB	A100-40GB	H100 SXM	B200 (approx.)
768	769	1,903	never	never
1,024	570	1,023	5,105	3,009
2,048	411	604	1,144	990
4,096	361	501	824	741
8,192	340	462	723	658
12,288	334	450	695	635

Three remarks. First, “tokens per weight visit” is the *micro-batch* on one device, in tokens: with gradient accumulation the weights are re-streamed every micro-batch, so T is micro-batch $\times$ sequence length, not the global batch. Tensor parallelism leaves $T_{\min}$ unchanged (per-device FLOPs and per-device weight bytes shrink together). Second, for any $d \geq 1024$ a single 2,048-token sequence already satisfies this floor on A100 and H100: *the sequence is the batch*, which is the transformer’s great advantage over an MLP on vectors, where T = the example count. Third, the floor is generation-dependent in the wrong direction: H100 needs 2.3 $\times$ the tokens A100 does at $d = 4096$, and its $d_{\min}$ has moved above GPT-2-medium.

Floor 2 — the small- d pole. $T_{\min}$ diverges at $d_{\min} = a\beta_t/72$ (447 on A100-80GB, 895 on H100). Below it the inter-kernel activation traffic alone exceeds the balance and *no batch size* exits the wall, because that traffic is proportional to T . The only levers are a (fusion, FP8 storage) and the ratio β_t (a slower, cheaper part). This is why GPT-2-small-sized models get 20–30% MFU on H100 no matter how they are batched.

Floor 3 — per-step fixed traffic. Terms (e) and (f) are paid once per optimizer step, not per micro-batch:

	FLOP/param				<10%
traffic	bytes/param	per step	intensity	parity threshold	overhead
Adam update (HBM)	30	$6 T_{\text{step}}$	$T_{\text{step}}/5$	$5\beta_t \approx 800$	$\approx 8,000$
DP ring all-reduce (fabric)	4	$6 T_{\text{dev}}$	$1.5 T_{\text{dev}}$	$\beta_{\text{net}}/1.5$: 700 NVLink, 8,300 IB HDR	7,000 / 83,000
ZeRO-3 / FSDP (fabric, per micro-batch)	6	$6 T_{\text{dev}}$	T_{dev}	β_{net} : 1,000 NVLink, 12,500 IB	10,000 / 125,000
TP activation all-reduce (fabric)	—	—	$4.5 d/p$	$p \leq 4.5d/\beta_{\text{net}}$ (≈ 18 at $d = 4096$, NVLink)	batch- independent
PP stage boundary (fabric)	—	—	$18 dL/p$	never binding	batch- independent

with β_{net} = peak/per-direction fabric bandwidth: 1,040 FLOP/B for NVLink-3 (300 GB/s per direction), 12,500 for one HDR NIC (25 GB/s). The last two rows are the escape hatch: tensor and pipeline parallelism add devices without adding batch, because their communication scales with d , not T . Data parallelism is the only axis whose communication demands batch, which is why the batch-size question is really a data-parallel-width question.

The ceiling — critical batch size. Beyond the gradient-noise scale, batch stops buying steps: McCandlish et al. 2018 give $S = S_{\min}(1 + B_{\text{noise}}/B)$ steps and $E = E_{\min}(1 + B/B_{\text{noise}})$ tokens; Kaplan et al. 2020 fit $B_{\text{crit}}(L) = B^*/L^{1/\alpha_B}$ with $B^* \approx 2 \times 10^8$ tokens and $\alpha_B \approx 0.21$, i.e. $B_{\text{crit}} \approx 0.5\text{--}4 \times 10^6$ tokens at the losses where pre-training lives (production runs use 2–16 M). So the memory-wall-free window is roughly

$$\underbrace{N \times \max(T_{\min}, T_{\text{fixed}})}_{\text{hardware floor}} \lesssim B \lesssim \underbrace{B_{\text{crit}}}_{\text{statistics}} ,$$

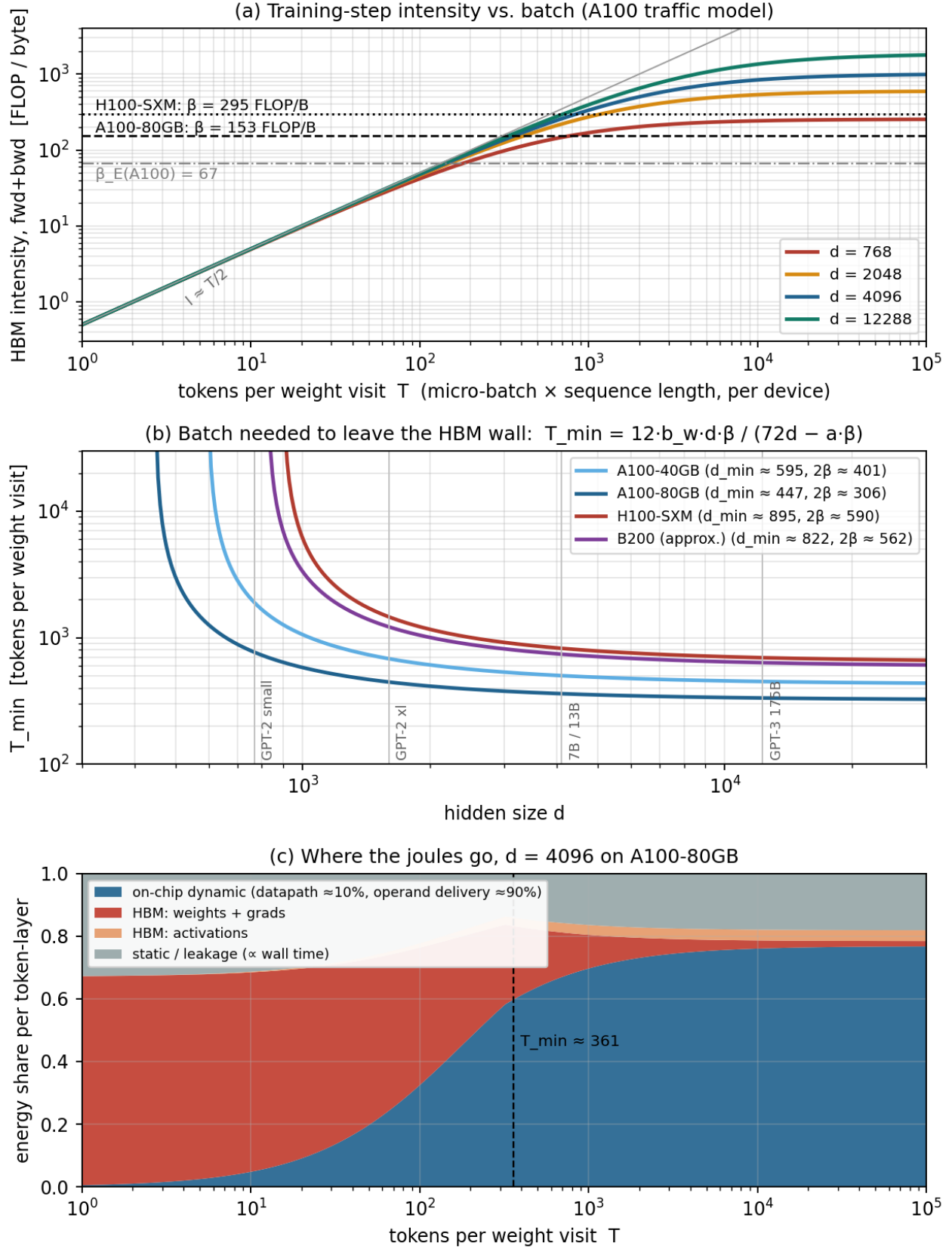


Figure 1: (a) HBM intensity of forward + backward versus tokens per weight visit; (b) minimum tokens per weight visit versus hidden size for four GPUs; (c) energy breakdown per token-layer versus batch for $d = 4096$ on A100-80GB.

about two orders of magnitude wide for a 7B model on a few hundred GPUs, and it closes as N grows or d shrinks.

The squeeze. Put the fixed cost and the ceiling in one objective. Per-device step time is $t = 6P T_{\text{dev}}/\text{peak} + t_{\text{fixed}}$; write $T_{\text{fixed}} = t_{\text{fixed}} \text{peak}/6P$ (the fixed cost in token-equivalents) and $B = NT_{\text{dev}}$. Total time $S(B)t(B)$ is minimized at

$$B_{\text{opt}} = \sqrt{B_{\text{crit}} N T_{\text{fixed}}}, \quad T_{\text{dev,opt}} = \sqrt{B_{\text{crit}} T_{\text{fixed}}/N}.$$

Table 4 — the squeeze for a 7B model, $B_{\text{crit}} = 10^6$, no communication overlap.

fabric	N	T_{fixed} (tok-eq/GPU)	B_{opt} (M tokens)	$T_{\text{dev,opt}}$	comm parity at
NVLink 300 GB/s	8	702	0.07	9,370	702
NVLink 300 GB/s	1,024	693	0.84	823	693
IB HDR 25 GB/s	64	8,202	0.72	11,321	8,202
IB HDR 25 GB/s	256	8,290	1.46	5,691	8,290
IB HDR 25 GB/s	1,024	8,313	2.92	2,849	8,313

At 1,024 GPUs over InfiniBand the time-optimal per-device batch (2,849 tokens) is *below* the communication-parity point (8,313): the optimizer deliberately operates inside the wall because the alternative, pushing B past B_{crit} , wastes samples faster than the wall wastes seconds. This is a sub-optimization result in your sense: a hardware team maximizing MFU and a research team maximizing sample efficiency pull toward different B , and the joint optimum is the geometric mean of their two targets. Overlap (the usual fix) reduces T_{fixed} but does not change the form.

4. What is specifically backprop’s fault

(a) Weight-side: not much. The backward pass reads W once more for ∇X and reads X and ∇Y for ∇W ; it triples the FLOPs and roughly doubles the bytes, so per pass its intensity is the same T FLOP/B as the forward pass. Backprop does not make the matmuls memory-bound. Its weight-side floor is twice that of inference decode only because of the fp32 gradient read-modify-write (the “12” in b_w), which is a choice of numerics, not of algorithm; with bf16 gradients the floor is β_t , identical to decode.

(b) Temporal non-locality: the real thing. The forward activation of layer ℓ is produced at time t_ℓ and consumed at $t_{2L-\ell}$. On a time-multiplexed machine that is a capacity cost of $34 d \cdot T \cdot L$ bytes per micro-batch (9.1 GB per 2,048-token sequence for a 32-layer $d = 4096$ model; 73 GB for eight of them, which is why eight sequences become eight micro-batches and T per weight visit stays 2,048), and it is what checkpointing and reversibility address. On a spatial machine it is an in-flight cost, Section 6.

(c) The store-versus-recompute energetics. Storing and reloading a token-layer’s activations costs $2 \times 34 d \times e_B = 4.1 d$ nJ; recomputing the forward costs $24 d^2 \times e_F = 21.6 d^2$ pJ. They cross at $d \approx 190$; above it storing is cheaper. Full checkpointing (store only the $2d$ layer input, recompute the rest) therefore costs +33% arithmetic energy for a $17\times$ capacity reduction, and lowers HBM *traffic* not at all; selective recompute (Korthikanti’s attention-only variant) gets most of the capacity at $\sim 3\%$ arithmetic. FlashAttention is the case where recompute wins on traffic: the $s \times s$ score matrix has $O(1)$ intensity through HBM, so recomputing it in shared memory during the backward pass ($\approx 4sd$ extra FLOPs per token) removes $O(s^2)$ bytes.

(d) The rank- T argument. $\nabla W = X^\top \nabla Y$ has rank $\leq T$. A micro-batch’s update to a $d \times k$ matrix contains at most $T(d+k)$ independent numbers, yet a time-multiplexed implementation

moves dk numbers of W (twice) and dk numbers of ∇W (read-modify-write) to apply it. The bytes are $dk/(T(d+k)) \approx 0.8d/T$ times redundant. At $T = 1$ and $d = 4096$ that is a $3,300\times$ overhead in bytes per unit of information, which is the entire per-token memory wall stated without reference to any bandwidth number. Weight-stationary arrays, LoRA-style factored updates and GaLore all exploit the same fact; the factors are cheaper to move than the product until $T \approx d$.

(e) The time tax and why fusion, not batch. The inter-kernel traffic ad per token is T -proportional, so it is a fixed *fraction* of step time, $a\beta_t/72d$, regardless of batch: 10% at $d = 4096$ and 55% at $d = 768$ on A100 (twice that on H100). During those seconds the tensor cores idle but the leakage clock runs, which is the $\sim 20\%$ static slice in Figure (c). Batching cannot shrink this slice; kernel fusion, FP8 activation storage, and reversibility can.

5. Single-example backprop, quantified

Table 5 — 7B-class model ($d = 4096$, $L = 32$, $s = 2048$, 6.85 B params) on A100-80GB; no communication overlap; GEMMs at peak (multiply the arithmetic time by 1/MFU for a real stack). Tokens per step are per GPU.

scenario	tok/step	ms/tok	mJ/tok	arith %	HBM %	static %	I FLOP/B	slowdown
online, 1 token/step, 1 GPU	1	138.7	25,337	0.1	67.0	32.9	0.15	1,034×
1 sequence/step, 1 GPU	2,048	0.196	60.1	62.8	17.6	19.6	237	1.46×
1 seq/GPU, 8 GPUs, NVLink DP	2,048	0.192	54.5	69.1	9.8	21.1	472	1.43×
1 seq/GPU, 64 GPUs over IB, DP	2,048	0.675	82.8	45.5	5.6	48.9	539	5.03×
8 seq/GPU (grad-accum), 64 GPUs IB	16,384	0.213	55.1	68.5	8.3	23.2	548	1.59×
64 seq/GPU (grad-accum), 64 GPUs IB	131,072	0.155	51.6	73.1	8.9	18.1	549	1.16×

Your intuition is right in the limit and needs one refinement in the middle:

- **Per token** (online learning, RL with per-step updates, streaming): catastrophic. Every layer streams 2.4 GB of weights and gradient state for 1.2 GFLOP of work; the step is $1,000\times$ slower than compute-bound and $500\times$ more energy per token than the batched regime. Nothing about this improves with the GPU generation, because β_t grows.
- **Per sequence, one GPU:** the sequence supplies $\sim 2,000$ tokens of weight reuse, so the GEMMs sit above β_t ($I = 237$). What is left is the per-step fixed traffic (205 GB of optimizer state per step against 0.27 s of compute), a $1.46\times$ slowdown and ~ 60 mJ/token. Note also that Adam’s state alone (110 GB) does not fit, so single-sequence training of a 7B model on one A100 is a capacity problem before it is a bandwidth one.
- **Per sequence, many GPUs:** the DP all-reduce of 27 GB of gradients per step over a 25 GB/s NIC costs 1.1 s against 0.27 s of compute: $5\times$ slowdown, half the energy is leakage while waiting. This is the regime in which “single example backprop is inefficient” is most true in practice, and it is a fabric wall, not an HBM wall.
- **Batched:** 8 sequences per GPU recover to $1.6\times$, 64 to $1.16\times$; the remaining 16% is the element-wise time tax plus the $\sim 10\%$ per-step terms. Energy floors at ~ 50 mJ/token with 73% in on-chip dynamic power, of which roughly a tenth is the actual multiply-adds.

For contrast: a CNN gets its reuse from spatial positions (a 224^2 image supplies $\sim 50k$ positions per convolution weight) and is fine at batch 1; an MLP on vectors is the worst case, with T equal to the example count; decode-time inference is the transformer’s own worst case, because the KV cache is per-sequence and batching does not amortize it. Dally’s Megatron-20B slide shows exactly that: a hypothetical Hopper with $10\times$ the HBM bandwidth is still $3.6\times$ faster at batch 256, so even large-batch decode remains bandwidth-bound.

6. On Dally’s grid

I could not fetch your repository (GitHub refuses automated access), so I use the model your directory name points to: the *spatial computer* of Gianinazzi, Ben-Nun, Besta, Ashkboos, Baumann, Luczynski & Hoefer (2022), in which processors sit on a 2D grid, the energy of a computation is the total byte-distance travelled by all messages, and the depth is the longest chain of hops; a variant adds a local-memory size parameter. Map “hop” to your model’s unit; the ratios below do not depend on it.

Forward pass. Put a $d \times k$ layer, weight-stationary, on a $\sqrt{n} \times \sqrt{n}$ block of tiles, each holding a $(d/\sqrt{n}) \times (k/\sqrt{n})$ slice. Per token, the $2d$ input bytes travel across a row of $\sqrt{n}$ blocks and the $2k$ output bytes (partial sums) travel down a column of $\sqrt{n}$ blocks: $2(d+k)\sqrt{n}$ byte-hops for $2dk$ FLOPs, i.e.

$$\text{hops per FLOP} = \frac{(d+k)\sqrt{n}}{dk} \approx \frac{1.25\sqrt{n}}{d} \quad (k = 4d).$$

For $d = 4096$, $k = 16384$ on $n = 256$ tiles (0.5 MB of weights each): 4.9×10^{-3} byte-hops per FLOP, which at 0.5 mm pitch (0.4 pJ per byte-hop, 50 adds) is 0.002 pJ/FLOP, 2% of the datapath. Spatially the matmul is not the problem, and this is batch-independent: hops per FLOP depend on array geometry, not on T .

Backward pass. $\nabla X = \nabla Y W^\top$ is the same array with the flow reversed: ∇Y enters where Y left, partial sums exit where X entered. The transpose costs zero hops; that is the single biggest structural advantage a spatial layout has over a time-multiplexed one for backprop. $\nabla W = X^\top \nabla Y$ is an outer product, and each tile already sees exactly the slices of X and ∇Y it needs pass through it. So ∇W costs no hops *if each tile remembers its slice of X until ∇Y arrives*: storage of $T \cdot d/\sqrt{n}$ elements per tile, i.e. the layer input replicated $\sqrt{n}$ times across the column blocks. The alternative, storing X once at the layer boundary and re-broadcasting it when ∇Y comes back, costs a second $2d\sqrt{n}$ byte-hops per token. Either way the backward pass pays a $\sqrt{n}$ factor that the forward pass did not, in storage or in hops.

In-flight storage: the L^2 . A token that passes layer ℓ at time $\ell\tau$ returns to it at $(2L-\ell)\tau$. With the pipeline full, the tokens layer ℓ must hold number $2(L-\ell)$ times the tokens admitted per layer-latency, so the total in-flight activation storage is

$$\sum_{\ell} 2(L-\ell) T_{\tau} c d \approx L^2 T_{\tau} c d \quad \text{bytes,} \quad c = 34 \text{ (store all) or } 2 \text{ (store inputs, recompute),}$$

where T_{τ} is the tokens in flight per layer. On a GPU pipeline with $p \ll L$ stages the same sum gives $\frac{1}{2} L p T_{\tau} c d$ (the 1F1B schedule of GPipe/PipeDream); a fully spatial machine is $p = L$. For $L = 32$, $d = 4096$, $T_{\tau} = 256$: 37 GB store-all, 2.1 GB inputs-only, versus 4.6 GB for the same model on an 8-stage GPU pipeline. Forward-only inference and any local learning rule have L , not L^2 . **This quadratic in depth, not the matmul hops, is the memory wall backprop hits on a Dally grid**, and it is why wafer-scale designs invert the mapping (Cerebras streams

weights past resident activations one layer at a time, which removes the L^2 and reinstates the “batch $\geq \beta$ of the weight fabric” rule instead).

Batch on the grid. Since hops per FLOP are batch-independent, batch matters on the grid only through local weight reuse: reading a weight from a tile’s SRAM costs ~ 1 pJ/byte, 5–10 $\times$ a MAC, so weights must be held in registers and applied to $T_{\text{local}} \geq e_{\text{SRAM}}/e_{\text{MAC}} \approx 10$ inputs before being refetched. This is the TPU’s weight-stationary systolic dataflow, and it is the same reuse rule as heuristic 1 with $\beta_{\text{SRAM}} \approx 10$ instead of $\beta_{\text{HBM}} \approx 150$. The grid converts a batch problem into a register-file problem.

What to count. For a hop-counting comparison of algorithms on the grid, four quantities capture the above: forward byte-hops, backward byte-hops (including any X re-broadcast), the in-flight byte-seconds $\sum_{\ell} 2(L - \ell) \tau$ (bytes stored at ℓ), and the depth $2L\sqrt{n}$. Backprop’s signature is a factor ~ 2 on the first two, $\sqrt{n}$ on the second, and L on the third relative to the forward pass.

7. What “solving it at prescribed accuracy” would take

The analysis says the wall has three separable parts, and existing proposals each attack one:

approach	capacity (34 d T L)	HBM / hop traffic	extra arithmetic	accuracy status
batching to $T \geq 2\beta_t$, $B \geq NT_{\text{fixed}}$	worse ($\propto T$)	fixes weight streaming and per-step terms	none	free below B_{crit} ; the squeeze above
kernel fusion, FP8 activations	$\sim 1/2$	fixes the $a d$ term; moves d_{min} down	none	FP8 activations are standard at scale
gradient checkpointing (Chen 2016; revolve, Griewank-Walther 2000)	$\sqrt{L}$ or $\log L$	unchanged or slightly up	+33% (full), $\sim 3\%$ (selective)	exact
reversible layers (Gomez et al. 2017; Rev-ViT)	$O(1)$ per layer; deletes the grid’s L^2	removes the activation round trip	+33%	matches at ViT/LM scale with architectural constraints
model parallelism (TP/PP/SP) instead of DP width	divides by p	fabric intensity $\propto d/p$, batch-free	none	exact; bounded by $p \leq 4.5d/\beta_{\text{net}}$
memory-light optimizers (8-bit Adam, Adafactor, GaLore)	—	$b_{\text{opt}} 30 \rightarrow 6\text{--}10$	none	small, model-dependent losses
synthetic gradients / DNI (Jaderberg 2017); pipelined backprop with stale weights (PipeDream)	breaks the L^2 into $L \times$ (module depth)	fewer hops	small	approximate; stale-gradient bias grows with depth and lr

approach	capacity (34 d T L)	HBM / hop traffic	extra arithmetic	accuracy status
local losses / greedy layer-wise / forward-forward (Nøkland & Eidnes 2019; Belilovsky 2019; Hinton 2022)	L	forward-only hops	1-2× forward	gap to backprop widens with scale
forward-mode / forward gradients (Baydin 2022; Ren 2023)	$O(1)$	forward-only	1 forward per direction	variance $\propto$ number of perturbed parameters; needs local-loss structure to be competitive
zeroth-order (MeZO, Malladi 2023)	inference memory	forward-only	2 forwards per step	10-100× more steps; energy per unit accuracy is worse

The honest summary for your problem statement: with backprop kept exact, the wall is *already* beatable in bandwidth and energy terms for $d \gtrsim 1000$ by (i) $T \geq$ a few hundred tokens per weight visit, which a single sequence supplies, (ii) fusion and low-precision storage for the ad term, (iii) model parallelism to add devices without adding batch, and (iv) reversibility or recompute for capacity. What no exact method removes is the per-step fixed traffic (which is why the squeeze exists) and the interior wall inside the FLOP. The approximate methods that remove the L^2 outright have not yet met “prescribed accuracy” at scale; the cleanest candidate that is both exact and spatially local is the reversible network, which turns backprop into a second, backwards forward pass and makes every layer’s storage $O(1)$ in depth.

8. Measuring it

On the A100 with NVML. Use `nvmlDeviceGetTotalEnergyConsumption` (a millijoule counter, Volta and later), not `nvmlDeviceGetPowerUsage`: the power reading on A100 is a windowed average that undersamples kernels shorter than a few hundred milliseconds, so repeat each kernel for ≥ 1 s and difference the counter. Two calibration kernels give you the constants of this report directly:

- a bandwidth-bound kernel (a large element-wise op or a device-to-device copy, ~ 2 TB/s): $(P_{\text{kernel}} - P_{\text{idle}})/\text{bytes per second}$ is the all-in HBM energy per byte; the model predicts ~ 120 W above idle, i.e. ~ 60 pJ/B, and any gap from the 40 pJ/B interface figure is the on-die transport you have been trying to reconcile with Dally’s wire numbers;
- a large BF16 GEMM at ~ 290 TFLOP/s: $(P - P_{\text{idle}} - P_{\text{HBM}})/\text{FLOP/s}$ is the all-in energy per FLOP, predicted ~ 0.9 pJ.

The signature to look for in a training step is bimodal power: ~ 380 – 400 W during GEMMs, ~ 200 – 250 W during the element-wise and optimizer phases. The fraction of time in the low mode is the time tax of heuristic 5; the energy in it is the static slice of Figure (c).

On the grid. Count separately the four quantities of Section 6 and report backprop as ratios to the forward pass: byte-hops (expect $\sim 2\times$, plus the $\sqrt{n}$ re-broadcast if you choose not to replicate X), in-flight byte-seconds (expect $\sim L\times$), and depth ($2\times$). A candidate algorithm “solves the problem” in the sense of your prompt if it brings the in-flight term from L^2 to L without raising the byte-hop term by more than the +33% that reversibility costs in arithmetic.

Appendix — parameters and their provenance

symbol	default	source / uncertainty
wire energy	100 fJ/bit·mm	Dally, AHA retreat keynote, Aug 2023
add energy	1 fJ/bit	same; “an add is worth 10 μm ”
small SRAM	50 fJ/bit	same
HBM access, interface	5 pJ/bit = 40 pJ/B	same slide (16 mm DRAM-die round trip is 1.6 pJ/b of it)
HBM all-in e_B	60 pJ/B	interface + ~ 25 mm on-die round trip at 100 fJ/b·mm; $\pm 30\%$
e_F all-in (A100)	0.90 pJ/FLOP	(400 – 60 – 40) W / 312 TFLOP/s; $\pm 20\%$; H100 0.55
e_{datapath}	0.10 pJ/FLOP	Dally’s 45 nm HMMA figure scaled; $\times 2$ uncertainty; the conclusions do not depend on it
static power	60 W (A100), 90 W (H100)	idle measurements
β_t	153 / 201 / 295 / 281	data-sheet dense BF16 peak $\div$ HBM bandwidth (A100-80/40, H100 SXM, B200 approx.)
b_w	12 B/param/micro-batch	2 bf16 reads + fp32 gradient RMW; lean stacks 6
a	200 d B/token/layer	kernel count, fused + FlashAttention; 120–300
saved activations	34 d B/token/layer	Korthikanti et al. 2022
Z	10^7 elements	half of L2 usable, bf16
b_{opt}	30 B/param/step	Adam mixed precision
fabric	300 / 25 GB/s per direction	NVLink-3 / one HDR NIC
B_{crit}	10^6 tokens	Kaplan et al. 2020 at pre-training losses

References. Dally, *Energy Efficiency and AI Hardware*, Stanford AHA retreat, 2023; Dally, “On the model of computation: point”, CACM 65(9), 2022; Williams, Waterman & Patterson, “Roofline”, CACM 2009; Hong & Kung, “I/O complexity: the red-blue pebble game”, STOC 1981; Korthikanti et al., “Reducing activation recomputation in large transformer models”, 2022; Dao et al., “FlashAttention”, 2022; Chen et al., “Training deep nets with sublinear memory cost”, 2016; Griewank & Walther, “Algorithm 799: revolve”, 2000; Gomez et al., “The reversible residual network”, 2017; Huang et al., “GPipe”, 2019; Narayanan et al., “PipeDream”, 2019; Rajbhandari et al., “ZeRO”, 2020; McCandlish et al., “An empirical model of large-batch training”, 2018; Kaplan et al., “Scaling laws for neural language models”, 2020; Gianinazzi et al., “The spatial computer”, 2022; Jouppi et al., “In-datacenter performance analysis of a TPU”, 2017; Jaderberg et al., “Decoupled neural interfaces using synthetic gradients”, 2017; Nøkland & Eidnes, “Training neural networks with local error signals”, 2019; Belilovsky et al., “Greedy layerwise learning can scale to ImageNet”, 2019; Hinton, “The forward-forward algorithm”, 2022; Baydin et al., “Gradients without backpropagation”, 2022; Ren et al., “Scaling forward gradient with local losses”, 2023; Malladi et al., “Fine-tuning language models with just forward passes”, 2023.